

TOAE209/TOAH209 – Design and Analysis of Algorithms

Week 14: Approximation Algorithms and Local Search

Foreign Trade University

Contents

1	Introduction: Coping with NP-Hardness	1
1.1	Approximation ratios	1
1.2	A spectrum of guarantees	2
2	Vertex Cover via Maximal Matching	2
3	The Greedy Set Cover Algorithm	3
4	Load Balancing (Makespan Minimization)	4
4.1	Longest-Processing-Time first (LPT)	5
5	Center Selection (k-Center)	5
6	Metric TSP via a Minimum Spanning Tree	6
7	Knapsack and an FPTAS	7
8	Local Search	7
8.1	Max-Cut local search	8
8.2	Escaping local optima: restarts and simulated annealing	8
9	Summary and Looking Ahead	9

1 Introduction: Coping with NP-Hardness

In earlier weeks we showed that many natural optimization problems are **NP-hard**: unless $P = NP$, no algorithm can solve them *exactly* in polynomial time on all inputs. But these problems do not go away – we still need to route vehicles, place facilities, schedule machines, and cover networks. This week we study two of the most important strategies for coping with NP-hardness while keeping a *provable* handle on solution quality:

- **Approximation algorithms** (Kleinberg & Tardos, Ch. 11): polynomial-time algorithms that come with a *guarantee* – the solution they return is always within a fixed factor of the optimum, on *every* input.
- **Local search** (Kleinberg & Tardos, Ch. 12): start with any feasible solution and repeatedly move to a “neighbouring” solution that improves the objective, until no improving move

remains. Sometimes a local optimum carries a provable approximation guarantee; often it is a powerful heuristic.

The intellectual shift from the greedy weeks is this: there, a clever local rule gave us the *exact* optimum, and the whole game was the correctness proof. Here, we accept that we will *not* get the optimum, and the game becomes proving how *close* we get. The central tool is the same in spirit as a greedy correctness proof, but turned into an inequality: we relate the algorithm's output to a *lower bound* (for minimization) or *upper bound* (for maximization) on the optimum that we can compute or reason about, even though we cannot compute the optimum itself.

1.1 Approximation ratios

Definition 1.1 (Approximation ratio). *Let Π be an optimization problem and A a polynomial-time algorithm for it. For an instance I , let $A(I)$ denote the value of the solution A produces and $\text{OPT}(I)$ the value of an optimal solution. We say A is a ρ -**approximation algorithm** (with $\rho \geq 1$) if:*

- for a minimization problem, $A(I) \leq \rho \cdot \text{OPT}(I)$ for every instance I ;
- for a maximization problem, $A(I) \geq \frac{1}{\rho} \text{OPT}(I)$ for every instance I .

The quantity ρ is the approximation ratio (or performance guarantee); smaller is better, and $\rho = 1$ means A is exact.

A recurring difficulty is that we cannot compare $A(I)$ to $\text{OPT}(I)$ directly – computing $\text{OPT}(I)$ is exactly the NP-hard problem we are avoiding. The trick that makes *every* proof this week work is to find a quantity $B(I)$ that we *can* reason about and that *bounds* the optimum:

$$B(I) \leq \text{OPT}(I) \quad (\text{lower bound, for minimization}),$$

and then prove $A(I) \leq \rho \cdot B(I)$. Chaining these gives $A(I) \leq \rho \cdot \text{OPT}(I)$ *without ever computing* OPT . Spotting the right bound B is the heart of the design.

1.2 A spectrum of guarantees

Not all approximation guarantees are equal:

- A **constant-factor** approximation has $\rho = c$ for a fixed constant (e.g. 2 for Vertex Cover, metric TSP, k -center, and load balancing).
- A **logarithmic** approximation has $\rho = O(\log n)$ (Set Cover); this is weaker, but for Set Cover it is essentially the best possible unless $P = NP$.
- A **PTAS** (polynomial-time approximation scheme) achieves $\rho = 1 + \varepsilon$ for any fixed $\varepsilon > 0$, in time polynomial in n (but possibly exponential in $1/\varepsilon$).
- An **FPTAS** (fully polynomial-time approximation scheme) achieves $1 + \varepsilon$ in time polynomial in *both* n and $1/\varepsilon$. Knapsack famously admits an FPTAS.

2 Vertex Cover via Maximal Matching

Definition 2.1 (Vertex Cover). *Given an undirected graph $G = (V, E)$, a vertex cover is a set $S \subseteq V$ such that every edge has at least one endpoint in S . The **Minimum Vertex Cover** problem asks for a cover of smallest size. (This problem is NP-hard.)*

A natural-sounding greedy rule – “repeatedly add the highest-degree vertex” – does *not* give a constant-factor guarantee (its ratio can grow like $\Theta(\log n)$). The following far simpler algorithm does.

Algorithm 1 Vertex Cover via Maximal Matching (Problem 1)

```

1:  $S \leftarrow \emptyset, M \leftarrow \emptyset$  ▷  $M$  is a matching
2: for each edge  $(u, v) \in E$  do
3:   if neither  $u$  nor  $v$  is in  $S$  then
4:      $S \leftarrow S \cup \{u, v\}$  ▷ add both endpoints
5:      $M \leftarrow M \cup \{(u, v)\}$ 
6:   end if
7: end for
8: return  $S$ 

```

The set M of edges we “commit to” is a *maximal matching*: a set of pairwise vertex-disjoint edges to which no further edge can be added. The cover S is exactly the set of endpoints of M , so $|S| = 2|M|$.

Theorem 2.2. *The algorithm returns a vertex cover S with $|S| \leq 2 \text{OPT}$, where OPT is the size of a minimum vertex cover. It is a 2-approximation.*

Proof. S is a cover. Suppose some edge (x, y) were uncovered, i.e. $x \notin S$ and $y \notin S$. Then when the loop processed (x, y) , neither endpoint was in S , so the algorithm would have added them – contradiction. Hence every edge is covered.

Lower bound on OPT . The edges of M are pairwise vertex-disjoint (once we add (u, v) , both endpoints are in S , so no later edge sharing u or v is added). *Any* vertex cover must contain at least one endpoint of each edge in M , and since the edges of M share no endpoints, these chosen endpoints are all distinct. Therefore every vertex cover – in particular the optimum – has size at least $|M|$:

$$\text{OPT} \geq |M|.$$

Combining. $|S| = 2|M| \leq 2 \text{OPT}$. □

The maximal matching M is precisely the computable *lower bound* B on OPT promised in the introduction. Problem 2 asks you to verify the factor of 2 empirically by computing the true optimum on small graphs by brute force.

Remark. It is a famous open-style fact that no polynomial-time algorithm is known with ratio better than $2 - o(1)$, and (under the Unique Games Conjecture) none can do better. The trivial-looking matching algorithm is, essentially, the best we know.

3 The Greedy Set Cover Algorithm

Definition 3.1 (Set Cover). *Given a universe $U = \{e_1, \dots, e_n\}$ of n elements and a collection of subsets $S_1, \dots, S_m \subseteq U$ whose union is U , a set cover is a sub-collection whose union is still all of*

U . The **Minimum Set Cover** problem asks for a cover using the fewest sets. (NP-hard; Vertex Cover is the special case where every element/edge lies in exactly two sets/vertices.)

Algorithm 2 Greedy Set Cover (Problem 3)

```

1:  $C \leftarrow \emptyset$  (chosen sets);  $R \leftarrow U$  (uncovered elements)
2: while  $R \neq \emptyset$  do
3:   Pick the set  $S_i$  maximizing  $|S_i \cap R|$  ▷ covers most uncovered elements
4:    $C \leftarrow C \cup \{S_i\}$ ;  $R \leftarrow R \setminus S_i$ 
5: end while
6: return  $C$ 

```

Let $H_n = \sum_{k=1}^n \frac{1}{k}$ denote the n -th harmonic number; recall $H_n = \ln n + O(1) = \Theta(\log n)$.

Theorem 3.2. *Greedy Set Cover returns a cover of size at most $H_n \cdot \text{OPT} = O(\log n) \cdot \text{OPT}$, where $n = |U|$ and OPT is the minimum number of sets in a cover.*

Proof. We use a *charging argument*. When the greedy algorithm picks a set S that covers t previously-uncovered elements, distribute a total “cost” of 1 (one set was used) evenly among those t elements: each newly-covered element e is charged $\text{cost}(e) = 1/t$. Each element is charged exactly once (when first covered), so the number of sets greedy uses equals $\sum_{e \in U} \text{cost}(e)$.

Key claim. For *any* set S_j in the optimal cover, the elements of S_j receive total charge at most $H_{|S_j|} \leq H_n$. To see this, order the elements of S_j by the time they were covered by greedy, say $e_1, e_2, \dots, e_{|S_j|}$ (the order in which greedy first covers them). Consider the moment just before greedy covers e_k . At that point, at least the elements $e_k, e_{k+1}, \dots, e_{|S_j|}$ of S_j are still uncovered – so S_j itself could cover at least $|S_j| - k + 1$ uncovered elements. Since greedy picks the set covering the *most* uncovered elements, the set S greedy actually picks covers at least $|S_j| - k + 1$ uncovered elements, so each gets charged at most $\frac{1}{|S_j| - k + 1}$. In particular $\text{cost}(e_k) \leq \frac{1}{|S_j| - k + 1}$. Summing over $k = 1, \dots, |S_j|$:

$$\sum_{e \in S_j} \text{cost}(e) \leq \sum_{k=1}^{|S_j|} \frac{1}{|S_j| - k + 1} = \sum_{i=1}^{|S_j|} \frac{1}{i} = H_{|S_j|} \leq H_n.$$

Combining. Let $\mathcal{O} = \{S_{j_1}, \dots, S_{j_{\text{OPT}}}\}$ be an optimal cover. Every element lies in at least one $S_j \in \mathcal{O}$, so

$$|\text{greedy cover}| = \sum_{e \in U} \text{cost}(e) \leq \sum_{S_j \in \mathcal{O}} \sum_{e \in S_j} \text{cost}(e) \leq \sum_{S_j \in \mathcal{O}} H_n = \text{OPT} \cdot H_n.$$

□

Problem 4 asks you to compare greedy’s cover size to the brute-force optimum on small instances and report the empirical ratio (which is typically much better than the H_n worst case).

4 Load Balancing (Makespan Minimization)

Definition 4.1 (Load Balancing). *We have m identical machines and n jobs with processing times $t_1, \dots, t_n$. Assign each job to a machine; the load L_k of machine k is the sum of the processing times of jobs assigned to it. The **makespan** is $\max_k L_k$, and we wish to minimize it. (NP-hard, even for $m = 2$ – it contains Partition.)*

Algorithm 3 Greedy List Scheduling (Problem 5)

```
1:  $L_k \leftarrow 0$  for all machines  $k = 1, \dots, m$ 
2: for each job  $j = 1$  to  $n$  (in the given order) do
3:   Assign job  $j$  to a machine  $k^*$  of currently minimum load
4:    $L_{k^*} \leftarrow L_{k^*} + t_j$ 
5: end for
6: return the assignment and makespan  $\max_k L_k$ 
```

We need two lower bounds on the optimal makespan OPT.

Lemma 4.2. $\text{OPT} \geq \frac{1}{m} \sum_j t_j$ (the average load) and $\text{OPT} \geq \max_j t_j$ (the largest single job).

Proof. The total work $\sum_j t_j$ is split over m machines, so some machine receives at least the average $\frac{1}{m} \sum_j t_j$; hence the makespan is at least that. And whichever machine runs the largest job has load at least $\max_j t_j$. $\square$

Theorem 4.3 (Graham, 1966). *Greedy list scheduling produces a makespan at most 2OPT : it is a 2-approximation.*

Proof. Let machine k achieve the final makespan $L = L_k$, and let job j be the *last* job assigned to k . Just before j was placed, machine k had load $L - t_j$, and – because greedy puts each job on a currently *least*-loaded machine – *every* machine had load at least $L - t_j$ at that moment. Summing over all m machines, the total work assigned so far was at least $m(L - t_j)$, so

$$\sum_i t_i \geq m(L - t_j) \implies L - t_j \leq \frac{1}{m} \sum_i t_i \leq \text{OPT}$$

by the first bound of Lemma 4.2. Also $t_j \leq \max_i t_i \leq \text{OPT}$ by the second bound. Adding,

$$L = (L - t_j) + t_j \leq \text{OPT} + \text{OPT} = 2 \text{OPT}.$$

$\square$

4.1 Longest-Processing-Time first (LPT)

A small change improves the guarantee: sort jobs in *decreasing* order before running greedy. The intuition is that big jobs, scheduled first, are balanced out by the many small jobs that follow.

Algorithm 4 Longest-Processing-Time First, LPT (Problem 6)

```
1: Sort jobs so that  $t_1 \geq t_2 \geq \dots \geq t_n$ 
2: Run Greedy List Scheduling on this sorted order
```

Theorem 4.4 (Graham, 1969). *LPT produces a makespan at most $\frac{4}{3} \text{OPT}$ (more precisely $\leq (\frac{4}{3} - \frac{1}{3m}) \text{OPT}$).*

Proof sketch. Re-run the analysis of Theorem 4.3 with the critical (last) job j on the bottleneck machine. If j is not the only job on its machine, then since jobs are sorted in decreasing order, j is among the smaller jobs; one shows that if $t_j > \frac{1}{3} \text{OPT}$, then in the optimal solution *every* machine runs at most two jobs, and a direct case analysis of two-jobs-per-machine optima shows LPT is in fact optimal in that regime. Otherwise $t_j \leq \frac{1}{3} \text{OPT}$, and the bound $L = (L - t_j) + t_j \leq \text{OPT} + \frac{1}{3} \text{OPT} = \frac{4}{3} \text{OPT}$ follows exactly as before. The full case analysis is in Kleinberg & Tardos, Section 11.1. $\square$

Problem 6 verifies the $\frac{4}{3}$ bound empirically on random instances against a lower bound, and Problem 5 verifies the 2-bound for plain greedy.

5 Center Selection (k -Center)

Definition 5.1 (k -Center). *Given a set of sites P in a metric space (distances satisfy the triangle inequality) and an integer k , choose a set $C \subseteq P$ of k centers to minimize the covering radius $r(C) = \max_{p \in P} \min_{c \in C} d(p, c)$ – the farthest any site is from its nearest center. (NP-hard.)*

Algorithm 5 Greedy Farthest-Point k -Center (Problem 7)

```

1: Pick any site  $c_1$ ; set  $C \leftarrow \{c_1\}$ 
2: for  $i = 2$  to  $k$  do
3:   Let  $p$  be the site farthest from  $C$ , i.e. maximizing  $\min_{c \in C} d(p, c)$ 
4:    $C \leftarrow C \cup \{p\}$ 
5: end for
6: return  $C$  and  $r(C)$ 

```

Theorem 5.2. *The greedy farthest-point algorithm is a 2-approximation: $r(C) \leq 2r^*$, where r^* is the optimal covering radius.*

Proof. Let $r = r(C)$ be the radius returned, achieved by some site p at distance r from its nearest chosen center. We exhibit $k + 1$ sites that are pairwise more than r apart (if $r > 2r^*$), which is impossible under k optimal centers.

Consider the k chosen centers together with p : that is $k + 1$ sites. Each chosen center c_i , at the moment it was selected, was the point farthest from the centers chosen before it – by the greedy rule, distances of newly selected centers to the current center set are non-increasing, and the final “leftover” distance is exactly r (the radius). It follows that every pair among these $k + 1$ sites is at distance at least r from one another.

Now suppose for contradiction $r > 2r^*$. By pigeonhole, two of these $k + 1$ sites, say a and b , are served by the *same* optimal center c^* – so $d(a, c^*) \leq r^*$ and $d(b, c^*) \leq r^*$. By the triangle inequality,

$$d(a, b) \leq d(a, c^*) + d(c^*, b) \leq 2r^* < r,$$

contradicting that all pairs are at distance $\geq r$. Hence $r \leq 2r^*$. □

The triangle inequality is essential: without a metric, no constant-factor approximation for k -center is possible. Problem 7 asks you to implement this and return both the centers and the radius.

6 Metric TSP via a Minimum Spanning Tree

Definition 6.1 (Metric TSP). *Given n cities and symmetric distances $d(i, j)$ satisfying the triangle inequality, a tour visits every city exactly once and returns to the start. The **Metric Traveling Salesman Problem** asks for a minimum-total-length tour. (NP-hard.)*

Theorem 6.2. *The MST-based tour has length at most $2w(T) \leq 2\text{OPT}$, where $w(T)$ is the weight of the MST. It is a 2-approximation for Metric TSP.*

Algorithm 6 MST-Based Metric TSP 2-Approximation (Problem 8)

- 1: Compute a minimum spanning tree T of the complete weighted graph on the cities
 - 2: Pick any root; list the vertices in the order of a depth-first *preorder* walk of T
 - 3: **return** the tour that visits the cities in that order (then returns to start)
-

Proof. Lower bound $w(T) \leq \text{OPT}$. Take any optimal tour and delete one of its edges: what remains is a path visiting all n cities, which is in particular a spanning tree. Its weight is at most the tour length, so the *minimum* spanning tree has weight $w(T) \leq \text{OPT}$.

Upper bound on the tour. Consider the “doubled” tree $2T$ (each tree edge duplicated). Every vertex now has even degree, so $2T$ has an Eulerian circuit – a closed walk traversing each edge exactly once, of total length exactly $2w(T)$. Walk along this Eulerian circuit and *shortcut*: whenever the walk would revisit an already-visited city, skip directly to the next unvisited city. This produces exactly the preorder visiting order. By the triangle inequality, each shortcut replaces a sub-walk by a direct edge of no greater length, so the resulting tour has length at most $2w(T)$.

Combining. tour $\leq 2w(T) \leq 2\text{OPT}$. $\square$

Remark. Christofides’s algorithm refines this to a $\frac{3}{2}$ -approximation by adding a minimum-weight perfect matching on the odd-degree vertices of T instead of naively doubling; we do not pursue it here, but it is the same MST-plus-shortcutting idea. Problem 8 asks you to verify tour $\leq 2w(T)$ directly.

7 Knapsack and an FPTAS

Definition 7.1 (0/1 Knapsack). *Given n items with profits $p_1, \dots, p_n$ and weights $w_1, \dots, w_n$, and a capacity W , choose a subset S with $\sum_{i \in S} w_i \leq W$ maximizing $\sum_{i \in S} p_i$. (NP-hard.)*

Knapsack has a dynamic program that runs in time $O(n \cdot P)$, where $P = \sum_i p_i$ is the total profit – *pseudo*-polynomial, since P can be exponential in the input size. The FPTAS controls this by *rounding and scaling* the profits so that P becomes small, at the cost of a controlled error.

Algorithm 7 Knapsack FPTAS (Problem 9)

- 1: **Input:** items (p_i, w_i) , capacity W , and $\varepsilon \in (0, 1)$
 - 2: Let $p_{\max} = \max_i p_i$ and set scaling factor $\mu = \frac{\varepsilon p_{\max}}{n}$
 - 3: Round each profit *down*: $\hat{p}_i = \lfloor p_i / \mu \rfloor$
 - 4: Solve Knapsack *exactly* on profits $\hat{p}_i$ (weights and W unchanged) by the profit-indexed DP
 - 5: **return** the chosen item set S
-

Theorem 7.2. *For any fixed $\varepsilon \in (0, 1)$, the algorithm runs in time polynomial in n and $1/\varepsilon$ and returns a set S with $\sum_{i \in S} p_i \geq (1 - \varepsilon) \text{OPT}$. It is an FPTAS.*

Proof sketch. Quality. Let O be an optimal set under the *true* profits. Rounding loses at most μ per item: $\mu \hat{p}_i \leq p_i \leq \mu \hat{p}_i + \mu$. The DP returns the optimum S under the rounded profits, so $\sum_{i \in S} \hat{p}_i \geq \sum_{i \in O} \hat{p}_i$. Multiplying by μ and accounting for the per-item rounding loss over the at-most- n items of O ,

$$\sum_{i \in S} p_i \geq \mu \sum_{i \in S} \hat{p}_i \geq \mu \sum_{i \in O} \hat{p}_i \geq \sum_{i \in O} p_i - n\mu = \text{OPT} - \varepsilon p_{\max}.$$

Since $\text{OPT} \geq p_{\max}$ (any single item fits, assuming $w_i \leq W$; otherwise discard oversized items first), $\varepsilon p_{\max} \leq \varepsilon \text{OPT}$, giving $\sum_{i \in S} p_i \geq (1 - \varepsilon) \text{OPT}$.

Running time. After scaling, every $\hat{p}_i \leq p_{\max}/\mu = n/\varepsilon$, so the total rounded profit is at most n^2/ε . The profit-indexed DP runs in $O(n \cdot \sum_i \hat{p}_i) = O(n^3/\varepsilon)$ – polynomial in both n and $1/\varepsilon$. $\square$

Problem 9 asks you to implement the scaled DP and verify value $\geq (1 - \varepsilon) \cdot \text{OPT}$ against the exact optimum on small instances.

8 Local Search

Local search abandons the “build a solution once and analyze it” template. Instead:

- (1) start from *any* feasible solution;
- (2) define a *neighbourhood*: the set of solutions reachable by a small modification (a “move”);
- (3) repeatedly move to a neighbour with strictly better objective value (this is *hill climbing*);
- (4) stop at a *local optimum*, where no neighbouring move improves.

The neighbourhood is the key design choice. A larger neighbourhood gives better local optima but costs more per step; a smaller one is cheaper but more easily trapped. For some problems a local optimum carries a *provable* approximation guarantee, as we now show for Max-Cut.

8.1 Max-Cut local search

Definition 8.1 (Maximum Cut). *Given an undirected graph $G = (V, E)$ with $m = |E|$ edges, partition V into two sides (A, B) to maximize the number of cut edges – edges with one endpoint in each side. (NP-hard.)*

Algorithm 8 Max-Cut Local Search (Flip a Vertex) (Problem 10)

- 1: Start from any partition (A, B) $\triangleright$ the neighbourhood: move one vertex to the other side
 - 2: **while** some vertex v has more edges to its *own* side than to the other side **do**
 - 3: Move v to the opposite side $\triangleright$ this strictly increases the cut
 - 4: **end while**
 - 5: **return** the partition and its cut value
-

Theorem 8.2. *Any local optimum of single-vertex flips cuts at least $\frac{m}{2}$ edges. Since $\text{OPT} \leq m$, this is a 2-approximation.*

Proof. For a vertex v , let $d_{\text{cut}}(v)$ be the number of its edges that cross the cut and $d_{\text{same}}(v)$ the number that stay on its side; these sum to $\deg(v)$. If we flipped v , the cut would change by $d_{\text{same}}(v) - d_{\text{cut}}(v)$. At a *local optimum*, flipping never helps, so for every v :

$$d_{\text{same}}(v) \leq d_{\text{cut}}(v), \quad \text{hence} \quad d_{\text{cut}}(v) \geq \frac{1}{2} \deg(v).$$

Summing over all vertices, each cut edge is counted twice (once per endpoint) in $\sum_v d_{\text{cut}}(v)$, and $\sum_v \deg(v) = 2m$:

$$2 \cdot (\text{cut size}) = \sum_v d_{\text{cut}}(v) \geq \frac{1}{2} \sum_v \deg(v) = \frac{1}{2} (2m) = m,$$

so the cut size is at least $m/2$. Finally $\text{OPT} \leq m$ trivially, so the local optimum is within a factor of 2 of optimal. $\square$

Each improving flip increases the (integer) cut value by at least 1, and the cut is bounded by m , so local search terminates after at most m flips. Problem 10 verifies the $\geq m/2$ guarantee, and Problem 11 compares hill-climbing (with random restarts) to the brute-force optimum on small graphs.

8.2 Escaping local optima: restarts and simulated annealing

Hill climbing gets *stuck* at the first local optimum it reaches, which may be far from global. Two standard remedies:

- **Random restarts.** Run hill climbing from many random starting points and keep the best local optimum found. Different starts fall into different “basins,” so more restarts increase the chance of finding the global optimum. Problem 11 explores this for Max-Cut.
- **Simulated annealing.** Borrowing from the physics of slowly cooling metals, occasionally accept a *worsening* move – with probability $e^{-\Delta/T}$, where $\Delta > 0$ is the amount the objective worsens and T is a “temperature” parameter – to escape local optima. The temperature starts high (almost any move accepted, exploring broadly) and is gradually lowered toward 0 (only improving moves accepted, settling into a good solution). With a slow enough cooling schedule it provably converges to the global optimum, though typically too slowly to be practical; in practice it is a robust, widely-used heuristic. Problem 12 implements a seeded simulated-annealing-style search for Max-Cut and checks that it never returns worse than a plain hill-climbing baseline on the tested instances.

This is the general theme of local search: a cheap, broadly applicable framework whose *worst-case* quality is hard to pin down in general, but which (a) sometimes admits clean guarantees, as for Max-Cut, and (b) is one of the most effective practical tools for large NP-hard optimization problems.

9 Summary and Looking Ahead

This week we studied two strategies for living with NP-hardness while keeping provable control over solution quality.

Approximation algorithms (with the lower-/upper-bound proof pattern):

- **Vertex Cover:** take both endpoints of a maximal matching; 2-approximation (Theorem 2.2).
- **Set Cover:** greedily cover the most uncovered elements; $H_n = O(\log n)$ approximation via a charging argument (Theorem 3.2).
- **Load Balancing:** greedy list scheduling is a 2-approximation (Theorem 4.3); sorting longest-first (LPT) improves it to $\frac{4}{3}$ (Theorem 4.4).
- **k -Center:** greedy farthest-point is a 2-approximation using the triangle inequality (Theorem 5.2).
- **Metric TSP:** MST preorder walk with shortcutting is a 2-approximation (Theorem 6.2); Christofides improves it to $\frac{3}{2}$.

- **Knapsack:** rounding-and-scaling profits yields an FPTAS – $(1 - \epsilon)$ of optimal in time polynomial in n and $1/\epsilon$ (Theorem 7.2).

Local search: hill climbing over a neighbourhood until no improving move remains. For Max-Cut, every single-flip local optimum cuts $\geq m/2$ edges, a 2-approximation (Theorem 8.2); random restarts and simulated annealing help escape poor local optima.

The unifying lesson is the **bounding technique:** we never compute OPT, yet we bound it – by a matching, by harmonic sums, by an average load, by an MST, by the triangle inequality – and squeeze the algorithm’s value against that bound.

Next week (*Randomized Algorithms*, Kleinberg & Tardos Ch. 13) we add a new resource – randomness – to the toolkit. We will see randomized contention resolution, the power of randomized rounding for approximation, a randomized min-cut algorithm, and how a coin flip alone gives a $\frac{1}{2}$ -approximation for Max-Cut *in expectation*, complementing this week’s deterministic local-search guarantee.